$ STOP RE-TEACHING YOUR REPO

Stop Re-Teaching Your Repo to Every Coding Agent.

Make context **portable**, **reviewable**, **fresh**, and **testable** before the agent edits. A concrete workflow for Cursor, Claude, Codex, Gemini, and OpenCode.

NAVIGATION TRACK | .AGENT-CONTEXT/ | 252 GRADED ANSWERS | 6 REPOS · 4 MODEL VARIANTS

SPEAKER

Amit Prusty

Cursor Meetup · Amsterdam · May 2026

REPO

github.com/cote-star/agent-context

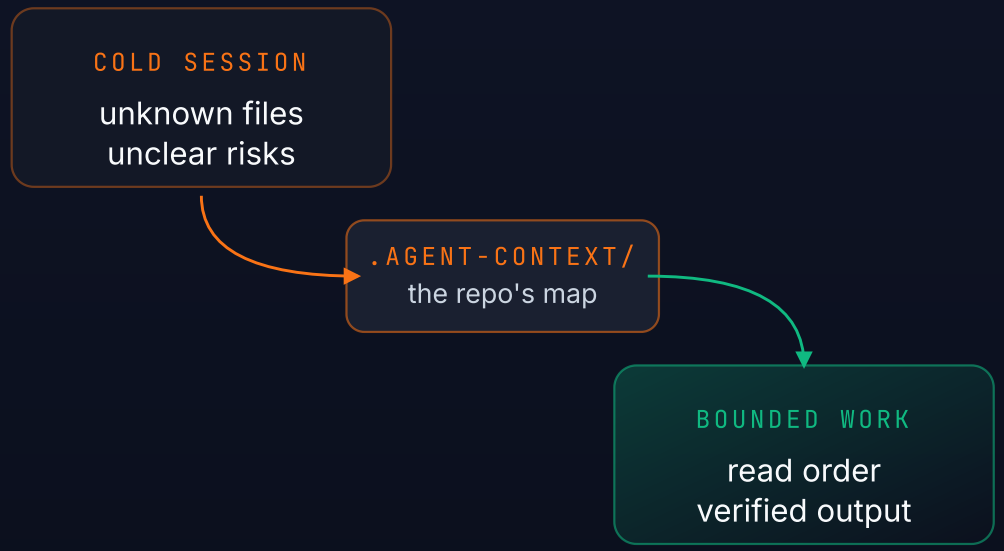
THE HOOK

The agent is new here every time.

Most coding agents enter a repo as a bright visitor with no durable map.

- They rediscover the tree.
- They infer ownership from whatever file they open first.
- They miss the invariant that decides whether an answer is safe.
- They repeat the same exploration in the next session, editor, or model.

> **A cold session** — unknown files, unclear risks, expensive search — gets converted into **bounded work** only when the repo carries its own map.



READ THE MAP BEFORE EDITING THE REPO

IT'S NOT THE MODEL

The failure is missing repo evidence.

We already check in tests, docs, schemas, CI, runbooks, and migration history.

Agent context should be another checked-in repo artifact: the map of what is true, risky, connected, and out-of-bounds.

PORTABLE

Markdown / JSON, not vendor memory.

REVIEWABLE

Lives in git. Reviewed like code changes.

FRESH

Drift is a failing preflight, not a quiet footnote.

THE STANDARD IS SIMPLE

Read the repo map before editing the repo.

PATTERN IS BIGGER THAN CODE

Every serious corpus needs a navigation layer.

The dual-routing architecture generalizes to any file system or stable corpus an agent must explore before acting:

01	02	03	04 ← TODAY
Stable datasets	Design systems	Operations runbooks	**Code repositories**

But today, we focus on where it hurts developers the most: **code repositories.**

THE ARCHITECTURE

Explorable repo recall as a three-track system.

GLOSS · Explorable recall = an agent can re-discover the same map any time, on its own, from checked-in repo artifacts.

agent-context works when **navigation**, **operating loop**, and **engineering** are treated as separate controls.

NAVIGATION

What should load, what should not, when should it stop?

control surface — read order / risky paths

OPERATING LOOP

How does this agent consume repo context during real work?

agent behavior — trust route / verify route

ENGINEERING

What makes prior work durable, inspectable, reusable?

systems layer — manifest / tests / CI

OUTCOMES WHEN ALL THREE LINE UP

bounded loading · provenance · faster evidence

TODAY'S SESSION

The Navigation track only.

Navigation = the control surface. What the agent loads, doesn't load, and when it stops.

WHAT TO LOAD

Completeness contracts that name every file family that must change together.

WHAT NOT TO LOAD

Search scopes that bound where the agent grep-walks.

WHEN TO STOP

Stop rules + verification shortcuts that say "you have enough to answer."

OUT OF SCOPE TODAY Operating Loop (how Claude vs Cursor vs Codex consume the same pack) and Engineering (manifest / tests / CI / freshness gates) — companion conversations.

THE ARTIFACT · REAL REPO

`.agent-context/` — a committed folder.

No hosted service. No hidden memory. No editor lock-in. Numbered files give the agent a deterministic read order.

.agent-context — cote-star/agent-context

- current — CONTENT · AUTHORITY · NAVIGATION
  - 00_START_HERE.md — ROUTING
  - 10_SYSTEM_OVERVIEW.md — CONTENT
  - 20_CODE_MAP.md — CONTENT
  - 30_BEHAVIORAL_INVARIANTS.md — CONTENT
  - 40_OPERATIONS_AND_RELEASE.md — CONTENT
  - acceptance_tests.md — QUALITY
  - completeness_contract.json — AUTHORITY
  - manifest.json — QUALITY
  - reporting_rules.json — POLICY
  - routes.json — AUTHORITY
  - search_scope.json — NAVIGATION
- tools — QUALITY
  - check_freshness.sh — QUALITY
  - verify_context_pack.py — QUALITY

CONTENT

`00_*` through `40_*` · markdown map.

AUTHORITY

`routes.json` · `completeness_contract.json` .

NAVIGATION

`search_scope.json` + verification shortcuts.

QUALITY

`manifest.json` · `acceptance_tests.md` · `tools/` .

CURSOR CLAUDE CODEX GEMINI OPENCODE

Tier 1 = code map + search scope, 2 files. **Tier 3** = full pack, 11 files. Start tier 1; promote when the repo has cross-cutting invariants.

BEHAVIOR

Same navigation design, opposite agent loops.

TRUST-AND-FOLLOW

Claude · Gemini

Reads the pack as authoritative. Stops when the contract says done.

14 → 4 files opened
claude bare → structured

Compressed search → fewer files opened.

SEARCH-AND-VERIFY

Cursor · Codex

Bounds its grep to scoped directories and cross-checks verification shortcuts.

3 → 12 files opened
cursor — each verified against shortcut

Expanded proof burden → more verification, fewer dead ends.

Model-agnostic by construction. The routing block — `CLAUDE.md` / `AGENTS.md` / `.cursorrules` — points tools at `.agent-context/` before source exploration.

A REPEATABLE LOOP

The engineering pipeline — live, on stage.

01 **Initialize** — scaffold the pack.

02 **Fill** — agent enumerates subsystems, fills templates (`SKILL.md`).

03 **Verify** — schema checks, grep-backed acceptance tests.

04 **Freshness** — fail the run when relevant code changed but context did not.

05 **Measure** — bare vs `structured_fresh` on hard tasks.

DEMO CHOREOGRAPHY · 85S TOTAL

BEAT 1 **live** `init` · ~40s
empty repo → run `agent-context init` ; show the generated routing block.

BEAT 2 **live** `fill` · ~30s
narrate: " `SKILL.md` drives the agent — inventory subsystems, fill 11 templates, write grep-backed tests."

BEAT 3 **live** `verify` · ~10s
machine checks pass — proof the generation worked.

```
$ agent-context init .
scaffold .agent-context/  ok
$ agent-context verify .
routes.json schema        ok
acceptance tests (12/12) ok
```

MEETUP TAKEAWAY

A repeatable operating loop, not a magic prompt.

QUALITY GATE

What a reviewer actually checks.

The review target is the context **diff**, not the model's private memory.

FILE FAMILIES

Does the pack name every file family that must change together?

NEGATIVE GUIDANCE

Does it say which plausible files are deprecated, generated, or near-misses?

SILENT FAILURES

Does it call out the thing that can break without a compile error?

```
// .agent-context/current/30_BEHAVIORAL_INVARIANTS.md
# DO NOT TOUCH — auth/session.ts uses a non-obvious cookie schema.
Changing this without updating cookie/v2 will fail silently in prod
(no compile error; sessions invalidate overnight). See routes.json#auth.session.
```

Pull-request-style review of the context diff is the durable quality gate.

TEST PROTOCOL

How to test whether it works.

The methodology is deliberately boring:

PROTOCOL PIECE	WHY IT MATTERS
bare clone vs structured_fresh clone	Same repo, same task, only context changes.
Hard multi-hop tasks	Lookup tasks prove little; synthesis exposes wrong turns.
Grep-backed ground truth	Every expected file family is mechanically checkable.
Independent judge + audit	Self-scores are evidence, not verdicts.

Freshness is a hard experiment gate. If the pack is stale, the run fails before agents start.

Q2 2026 MULTI-AGENT RERUN

Quantified evidence.

252 GRADED ANSWERS · 48 CELLS · 6 REPOS · 4 MODEL VARIANTS · BARE · STRUCTURED_FRESH

AGENT / MODEL	BARE	STRUCTURED	Δ
Claude Opus 4.7 Trust-and-Follow	80% (4.80/6)	100% (6.00/6)	+20pp
Cursor `claude-opus-4-7-medium`	89% (5.33/6)	97% (5.83/6)	+8pp
Cursor `composer-2-fast` default	61% (3.67/6)	81% (4.83/6)	+20pp
Codex CLI `0.130.0`	72% (4.33/6)	78% (4.67/6)	+6pp

CLAUDE OPUS + STRUCTURED

6/6 perfect across all 6 repos.

COMPOSER-2-FAST

+20pp — biggest absolute lift.

RISK → 0

Codex (0.33→0.00) · Cursor Opus med (0.50→0.00).

CURSOR OPUS MED

−65% duration (219s→78s).

* **Risk-flag** = the agent confidently cites the wrong file or misses an invariant; acting on the answer would break production. · LLM-provisional grading — see methodology slide.

PER-AGENT · TRUST-AND-FOLLOW

Claude Opus 4.7 — 6/6 perfect under structured.

CORRECTNESS

+20pp

80% → 100%

TOOL CALLS / CORRECT

-38%

14.2 → 8.8

FILES OPENED / TASK

-40%

4.5 → 2.7

Narrative. Claude consumes the pack as authoritative — opens fewer files; the top-level Grep tool is not invoked. Tool-call efficiency gain is the strongest of any lane.

REFERENCE	BARE	STRUCTURED	Δ
Risk-flag rate / 6 tasks	0.20	0.17	small
Median duration / task	65s	56s	-14%
Re-read rate same file twice	0.015	0.004	-73%
Citations / task	8.9	9.3	flat
Quality self-score	8.8	9.0	up
Dead ends / post-hit	0 / 0	0 / 0	perfect

6 cells / 36 graded tasks · agent-chorus/bare ran on Haiku (5h-session fallback) — reported separately.

PER-AGENT · SEARCH-AND-VERIFY

Cursor `claude-opus-4-7-medium`

MEDIAN DURATION

-65%

219s → 78s

RISK-FLAG RATE

0.50→0

eliminated

CORRECTNESS

+8pp

89% → 97%

Narrative. Bare Opus medium is the slowest lane — heavy `glob` + `grep` recon before reads. Hop-count up = a feature: structured Opus reads more files near the answer to verify pack vs source.

REFERENCE	BARE	STRUCTURED	Δ
Tool calls per correct	8.3	7.9	small
Files opened / task	4.8	3.9	-19%
Hops to first correct file verification depth	0.83	1.42	↑ feature
Search-vs-read ratio	0.89	0.57	dropped
Verification-shortcut hit rate	n/a	0.15	new
Dead ends / post-hit	0.1 / 0	0 / 0	eliminated

6 cells / 36 graded tasks · daemon-reference/bare hallucinated writes on first attempt; recovered on retry, 6/6 final.

PER-AGENT · DEFAULT MODEL

Cursor `composer-2-fast` — the pack rescues a weaker model.

CORRECTNESS

+20pp

61% → 81%

TOOL CALLS / CORRECT

-18%

5.6 → 4.6

RISK-FLAG RATE

0.67→0.50

still riskiest

Narrative. Smallest duration lift (none) but largest correctness lift. Still the riskiest lane in both conditions — structured doesn't eliminate composer risk flags entirely. Lower citation count throughout — composer cites less by design.

REFERENCE	BARE	STRUCTURED	Δ
Median duration / task	111s	112s	flat
Files opened / task	2.9	2.6	small
Search-vs-read ratio	0.52	0.44	down
Verification-shortcut hit rate	n/a	0.17	new
Citations / task	5.3	5.6	flat
Dead ends / post-hit	0.2 / 0	0.1 / 0	down

6 cells / 36 graded tasks · polyglot-monorepo/structured hit a transient provider error on first attempt; recovered on retry, 6/6 final.

PER-AGENT · SEARCH-AND-VERIFY

Codex CLI `0.130.0` — trades wall-clock for correctness.

PACK UTILIZATION

97%

0.14 → 0.97 · no skimming

RISK-FLAG RATE

0.33→0

eliminated

MEDIAN DURATION

55→126s

slower (honest)

Narrative. Codex is slower under structured — trades wall-clock for correctness. Pack utilization 97% means codex reads almost every file in `.agent-context/current/`. Verification-shortcut hit rate is the highest of any agent (19%).

REFERENCE	BARE	STRUCTURED	Δ
Correctness yes-rate	72%	78%	+6pp
Tool calls per correct	11.4	9.2	-19%
Files opened / task	6.2	5.1	-18%
Verification-shortcut hit rate	n/a	0.19	highest
Hops to first correct file verification depth	1.0	1.33	↑ feature
Quality self-score	8.9	9.3	up

6 cells / 36 graded tasks · all codex tokens are `cell_replicated` (per-cell session totals, not per-task).

TRUST THE NUMBERS

How we measured. What it covers. What it doesn't.

SCOPE

252 graded tasks · 48 cells · 6 repos × 4 model variants × 2 conditions × 6 tasks. Same template across all repos.

GRADING

Each cell graded by an independent Claude Code subagent — fresh context, fixed judge prompt. **No human spot-audit.** Treat numbers as directional, not certified.

PROTOCOL

Fresh-pack isolated. Every `structured_fresh` clone passed `agent-context verify` + `check_freshness.sh` before the agent started. Pack content authored at v0.2.0; validated at v0.3.1.

TELEMETRY CAVEATS

Cursor sessions in opaque SQLite — tokens/cost null in this rerun. Codex tokens are `cell_replicated` (per-cell session totals).

ANOMALIES PRESERVED · NOT MASKED

composer/structured/polyglot transient retry · opus-medium/bare/daemon hallucination retry · claude/bare/agent-chorus ran on Haiku via 5h-session fallback · `org-second-brain` skipped (pack/EXPERIMENT setup needs review). **Working slate is 6 repos.**

Public claims may only quote reviewer-confirmed rows. This rerun's grading is provisional; cell numbers may shift after spot-audit.

"ISN'T THIS JUST .CURSORRULES?"

How agent-context fits what you already do.

YOU ALREADY HAVE…	AGENT-CONTEXT
`.cursorrules` / `AGENTS.md` / `CLAUDE.md` single-file routing	Uses these as **routing blocks** pointing into the structured pack. **Not a replacement.**
MCP servers runtime tool access (web, DB, internal APIs)	Different layer. agent-context is the navigation map; MCP is access.
Vector RAG semantic recall over chunks	Pack is structural recall: deterministic file lists, contracts, verification shortcuts. **Pair them; don't pick.**
Cursor project memory / "rules for AI"	Pack supplements, doesn't replace. Pack lives in git; IDE memory may not.

For Cursor users specifically: `.cursorrules` already exists in your repo. agent-context adds the rest of the contract — completeness, search scope, verification shortcuts — and Cursor reads it natively.

NOT MAGIC

Tradeoffs.

MAINTENANCE COST

The pack must change when relevant code changes.

OVERFITTING RISK

Packs should route and bound search, not quote every answer.

REPO-SIZE FIT

Small repos may only need tier 1. Full packs scale to monorepos.

TELEMETRY GAPS

Cursor stores sessions in opaque SQLite; tokens/cost stay null without reverse-engineering.

HUMAN REVIEW REMAINS

Context improves first drafts; it does not replace code review.

RUNTIME COST

Structured runs are similar or cheaper per correct answer. Cost is human-time to author the pack.

THE BARGAIN

Spend a little repo-maintenance effort to **stop every agent from rediscovering the same map.**

TRY THIS ON ONE REPO

Tomorrow.

01 **Pick one painful workflow** — impact analysis, auth, cache, release, migration, or docs corpus.

02 **Create tier 1** — start with code map + search scope.

03 **Add one invariant** — name the silent failure and near-miss files.

04 **Run bare vs context** — same task, same repo, two clones.

05 **Review the diff** — files opened, dead ends, missing surfaces, risk.

That is enough to decide whether the repo needs the full tier 3 pack.

```
$ agent-context init .
$ agent-context verify .
$ agent-context freshness .
# 30 minutes. one repo. one workflow.
```

ANY AGENT THAT READS `AGENTS.MD`

...or runs in a repo can pick up the pack — Cursor, Claude Code, Codex, Gemini, OpenCode, plus JetBrains / VS Code Copilot.

$ MAKE CONTEXT PART OF THE REPO

Make context part of the repo.

Not a prompt trick. Not a vendor feature.
A checked-in standard teams can review.

INIT VERIFY FRESHNESS BARE VS STRUCTURED_FRESH

DO THIS WEEK

Pick one repo. Run `agent-context init`. Open a PR for the diff.

TODAY'S TRACK

Navigation. Operating Loop & Engineering = next conversations.

REPO · CONTACT

github.com/cote-star/agent-context

— Amit Prusty